ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ · ЛЕКЦИЯ 4

Численное решение дифференциальных уравнений

ОДУ, методы Эйлера и Рунге–Кутты, УЧП и метод конечных разностей

Лю Шисян

Кафедра теплофизики (Э6)
МГТУ им. Н.Э. Баумана
Москва / 2026

Место этой лекции в курсе



Лекция 4 — Численное решение дифференциальных уравнений

Рассмотрим охлаждение тела, обмен энергией между электронами и решёткой и теплопроводность в стержне.

Задачи занятия:

- решать начальные задачи методами Эйлера и Рунге–Кутты;
- строить разностные схемы для уравнения теплопроводности;
- задавать начальные и граничные условия;
- выбирать шаг по времени и проверять устойчивость.

План

1. Обыкновенные дифференциальные уравнения
2. Метод Эйлера
3. Методы Рунге–Кутты
4. Уравнения в частных производных
5. Метод конечных разностей для теплопроводности
6. Практика и итоги

1

Обыкновенные дифференциальные уравнения

Алгебраические уравнения и начальная задача

Алгебраическая задача

Находим отдельное неизвестное:

$$f(x) = 0,$$

или набор неизвестных:

$$A\mathbf{x} = \mathbf{b}.$$

Результат: число или вектор.

Начальная задача для ОДУ

Неизвестная величина изменяется во времени:

$$\frac{dy}{dt} = f(t, y)$$

Чтобы определить конкретное решение, задаём начальное состояние:

$$y(t_0) = y_0$$



Закон охлаждения Ньютона

Начальная задача

$$\frac{dT}{dt} = -k(T - T_\infty), \qquad T(0) = T_0.$$

Во всех примерах охлаждения:

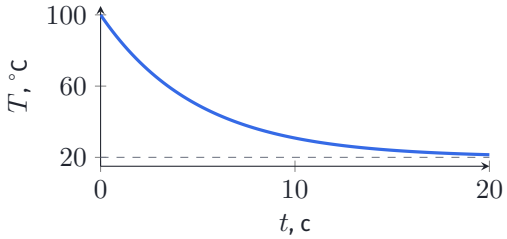
$$T_0 = 100\,^\circ\text{C}, \quad T_\infty = 20\,^\circ\text{C},$$

$$k = 0.2\ \text{s}^{-1}, \quad 0 \le t \le 20\ \text{с}.$$

Точную кривую используем для оценки ошибки.

Точное решение

$$T(t) = 20 + 80e^{-0.2t}.$$



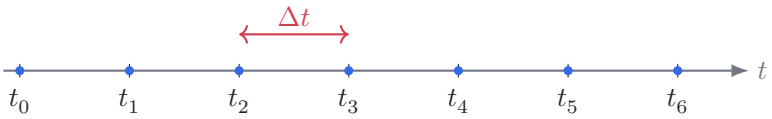
Дискретизация времени

Временная сетка

Непрерывную ось времени заменяем конечным набором узлов:

$$t_n = t_0 + n\Delta t, \qquad n = 0, 1, 2, \dots, N$$

где Δt — шаг по времени.



Численная задача

Вычисляем последовательность: $y_0,\ y_1,\ y_2,\ \dots,\ y_N$, где $y_n \approx y(t_n)$.

2

Метод Эйлера

Метод Эйлера

Исходная задача

Рассмотрим начальную задачу

$$\frac{dy}{dt} = f(t, y), \qquad y(t_0) = y_0.$$

На временной сетке известны

$$t_n, \qquad y_n \approx y(t_n).$$

Необходимо определить

$$y_{n+1}.$$

Метод Эйлера

Производную в точке t_n заменим разностью вперёд:

$$\left.\frac{dy}{dt}\right|_{t_n} \approx \frac{y_{n+1} - y_n}{\Delta t}.$$

Подставляем в ОДУ:

$$\frac{y_{n+1} - y_n}{\Delta t} = f(t_n, y_n).$$

Отсюда

$$\boxed{y_{n+1} = y_n + \Delta t\, f(t_n, y_n)}$$

Геометрический смысл метода Эйлера

В точке

$$(t_n, y_n)$$

производная

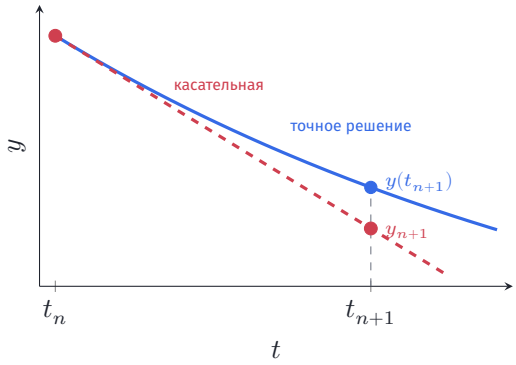
$$f(t_n, y_n)$$

задаёт наклон касательной.
Метод Эйлера предполагает, что на коротком интервале

$$[t_n, t_{n+1}]$$

решение можно заменить отрезком этой касательной.

$$y_{n+1} = y_n + \Delta t\, f(t_n, y_n)$$



Метод Эйлера для охлаждения

```python
import numpy as np

k, T_inf, T0 = 0.2, 20.0, 100.0

def euler_cooling(dt, t_end=20.0):
    n_steps = round(t_end / dt)
    t = np.linspace(0, t_end, n_steps + 1)
    h = t[1] - t[0]
    T = np.empty_like(t)
    T[0] = T0
    for n in range(n_steps):
        T[n+1] = T[n] - k*h*(T[n] - T_inf)
    return t, T
```

$$T_{n+1} = T_n - kh(T_n - T_\infty).$$

```python
t_euler, T_euler =
    euler_cooling(0.5)
print(T_euler[-1])
```

При $t = 20$ с

Эйлер: $21.18247\,^\circ\mathrm{C}$.

Точно: $21.46525\,^\circ\mathrm{C}$.

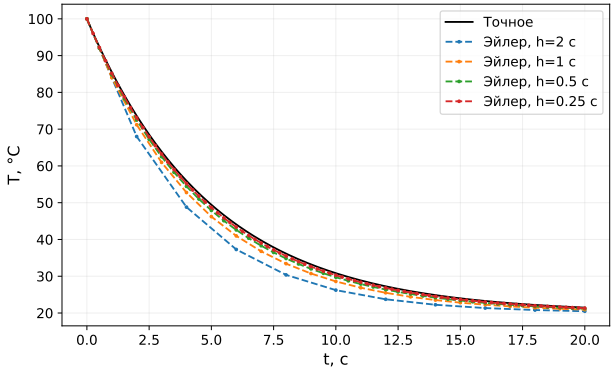
Фактический шаг h берём из временной сетки.

Влияние шага на решение

Один и тот же интервал
$0 \leq t \leq 20$ с.

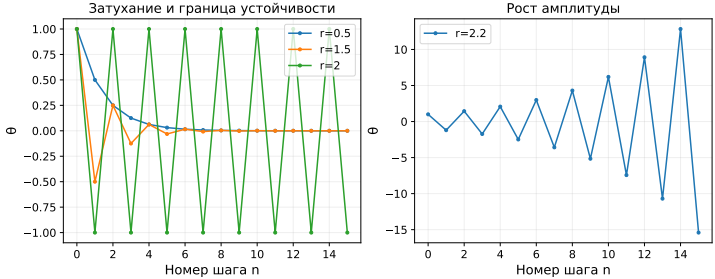
Уменьшаем шаг:
$2,\ 1,\ 0.5,\ 0.25$ с.

Сравниваем значения с точным решением в узлах каждой сетки.



Устойчивость метода Эйлера

$$\theta = \frac{T - T_\infty}{T_0 - T_\infty}, \qquad \theta_{n+1} = (1 - r)\theta_n, \qquad r = k\Delta t.$$



При $0 < r < 2$ амплитуда затухает; при $r = 2$ колебания сохраняются, при $r > 2$ растут.

При $r = 1$ схема обнуляет θ за один шаг. Устойчивость не означает точности.

3

Методы Рунге–Кутты

Наклоны внутри шага

Ограничение метода Эйлера

Метод Эйлера использует только наклон в начале шага:

$$k_1 = f(t_n, y_n).$$

Затем:

$$y_{n+1} = y_n + h\,k_1.$$

Если наклон внутри шага заметно меняется, такое приближение может быть неточным.

Методы Рунге–Кутты

Вместо одного наклона оцениваем **несколько наклонов внутри шага** и объединяем их.



Классический метод RK4

Для задачи

$$\frac{dy}{dt} = f(t, y)$$

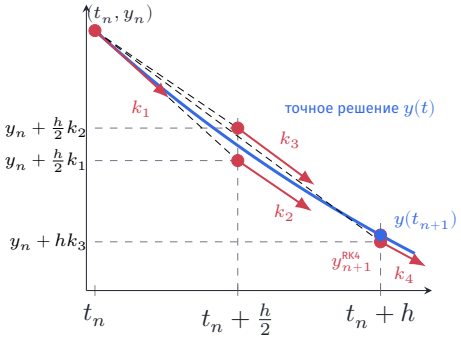
на одном шаге h вычисляются четыре наклона.

$$k_1 = f(t_n, y_n),$$

$$k_2 = f\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(t_n + \frac{h}{2},\, y_n + \frac{h}{2}k_2\right),$$

$$k_4 = f(t_n + h,\, y_n + hk_3).$$



После этого:

$$y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)$$

Метод RK4 на Python

Один шаг: четыре вычисления правой части

```
def rk4_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + h/2, y + h*k1/2)
    k3 = f(t + h/2, y + h*k2/2)
    k4 = f(t + h, y + h*k3)
    return y + h/6 * (k1 + 2*k2 + 2*k3 + k4)
```

```
def rhs_cooling(t, T):
    return -k * (T - T_inf)

T_next = rk4_step(rhs_cooling, 0.0, T0, 1.0)
```

После первого шага $h = 1$ с:

RK4: $85.49867\,^\circ\text{C}$.

Точно: $85.49846\,^\circ\text{C}$.

Расчёт всей траектории методом RK4

```
def rk4_cooling(dt, t_end=20.0):
    n_steps = round(t_end / dt)
    t = np.linspace(0, t_end, n_steps + 1)
    h = t[1] - t[0]
    T = np.empty_like(t)
    T[0] = T0
    for n in range(n_steps):
        T[n+1] = rk4_step(rhs_cooling, t[n], T[n], h
    )
    return t, T
```

```
t_rk4, T_rk4 =
    rk4_cooling(1.0)
print(T_rk4[-1])
```

При $t = 20$ с

RK4: $21.46534347\,^{\circ}\mathrm{C}$.

Ошибка: $9.24 \cdot 10^{-5}$ К.

Сходимость методов Эйлера и RK4

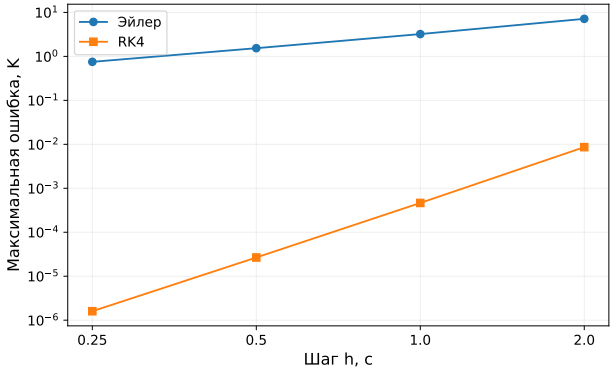
Ошибка на всей сетке:

$$E = \max_n |T_n - T(t_n)|.$$

При уменьшении шага вдвое:

Эйлер: примерно $E/2$.

RK4: примерно $E/16$.



Решение ОДУ с помощью SciPy

```python
from scipy.integrate import solve_ivp

t_eval = np.linspace(0, 20, 201)
sol_cooling = solve_ivp(
    rhs_cooling, (0, 20), [T0],
    t_eval=t_eval, method="RK45",
    rtol=1e-7, atol=1e-9)

T = sol_cooling.y[0]
print(sol_cooling.success)
```

Параметры

`y0`: начальное состояние.

`t_eval`: точки вывода.

`rtol`, `atol`: допуски для контроля локальной ошибки.

Вывод: `True`.

Внутренний шаг RK45 выбирает автоматически. Расстояние между точками `t_eval` не задаёт шаг интегрирования.

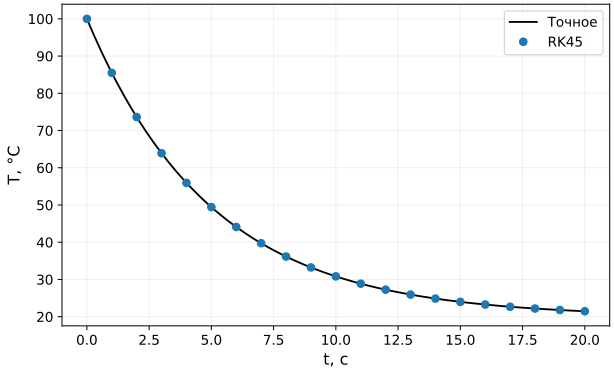
Проверка решения SciPy

Сравниваем с точным решением в 201 точке.

Максимальная ошибка:

$$E \approx 2.50 \cdot 10^{-6} \text{ к.}$$

`sol.success` проверяет завершение расчёта; ошибку оцениваем отдельно.



Системы ОДУ

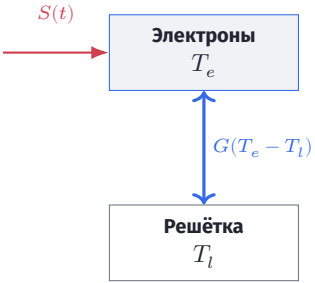
Пример: двухтемпературная модель

После быстрого возбуждения электроны и кристаллическая решётка могут иметь разные температуры:

$$C_e \frac{dT_e}{dt} = -G(T_e - T_l) + S(t)$$

$$C_l \frac{dT_l}{dt} = G(T_e - T_l)$$

- T_e, T_l — температуры электронов и решётки;
- C_e, C_l — объёмные теплоёмкости;
- G — коэффициент связи;
- $S(t)$ — внешний источник энергии.



$$\mathbf{y} = \begin{bmatrix} T_e \\ T_l \end{bmatrix}, \quad \frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y})$$

Двухтемпературная модель на Python

```
Ce, Cl, G = 1.0, 5.0, 0.8

def source(t):
    return 200.0 * np.exp(-(t - 1.0)**2 /
    0.1)

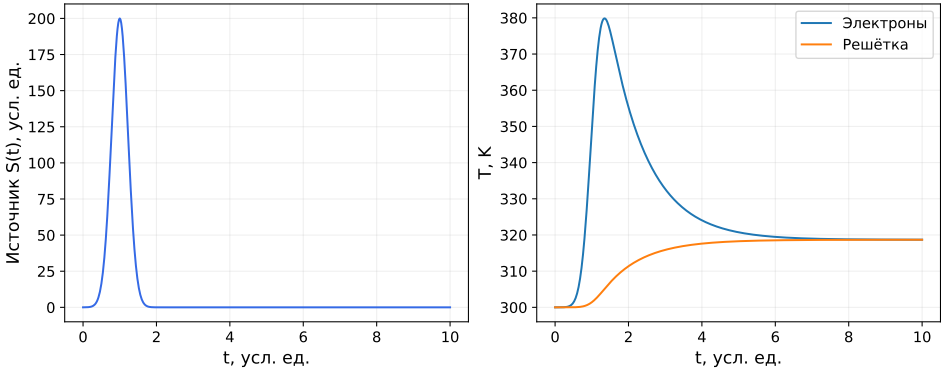
def rhs_ttm(t, y):
    Te, Tl = y
    q = G * (Te - Tl)
    return [(-q + source(t)) / Ce, q / Cl]
```

```
times = np.linspace(0, 10, 501)
sol_ttm = solve_ivp(
    rhs_ttm, (0, 10), [300.,
    300.],
    t_eval=times, method="RK45",
    rtol=1e-7, atol=1e-9,
    max_step=0.02)
Te, Tl = sol_ttm.y
```

`max_step` ограничивает внутренний шаг, чтобы разрешить короткий импульс.

Учебные параметры: время и энергетические коэффициенты в согласованных условных единицах, температура в К.

Импульсный нагрев электронов и решётки



Электроны нагреваются первыми. Затем энергия передаётся решётке.

Максимум T_e на сетке: 379.85 К. Предельная общая температура: 318.68 К.

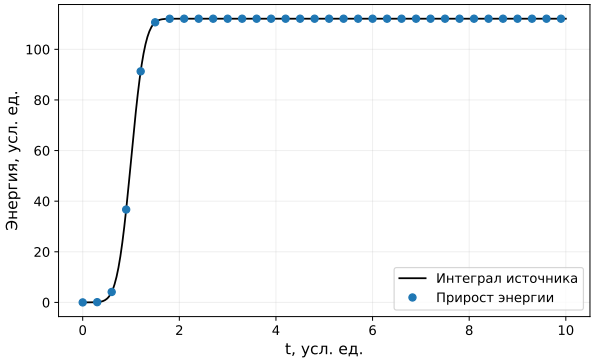
Проверка баланса энергии

При постоянных C_e, C_l и без теплообмена с внешней средой:

$$\frac{d}{dt}(C_e T_e + C_l T_l) = S(t).$$

Сравниваем прирост энергии с интегралом гауссова импульса, вычисленным через `erf`.

Максимальное расхождение: $1.19 \cdot 10^{-9}$ усл. ед.



4

Уравнения в частных производных

Температура тела и температурное поле

При равномерной температуре тела ищем одну функцию:

$$T = T(t).$$

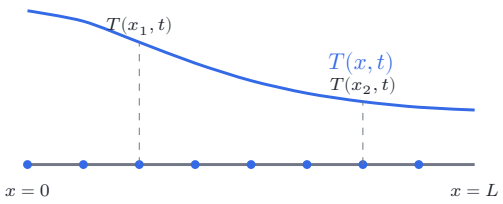
Её изменение описывает ОДУ.

Распределённая система

При неоднородном нагреве ищем температурное поле:

$$T = T(x, t).$$

Температура зависит и от координаты, и от времени.



В пространственно распределённой задаче появляются производные

$$\frac{\partial T}{\partial t}, \qquad \frac{\partial T}{\partial x}, \qquad \frac{\partial^2 T}{\partial x^2}.$$

Уравнение теплопроводности

Для однородного одномерного тела без объёмного источника тепла:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}$$

где $\alpha = k/(\rho c_p)$ — температуропроводность.

Производные температуры

$$\frac{\partial T}{\partial t}$$

показывает, как температура меняется во времени.

$$\frac{\partial^2 T}{\partial x^2}$$

характеризует локальную кривизну температурного профиля.

Начальные и граничные условия

Начальное условие

$$T(x,0) = T_0(x)$$

и **граничные условия** при $x = 0, \qquad x = L.$

Например:

$$T(0,t) = T_L, \qquad T(L,t) = T_R.$$

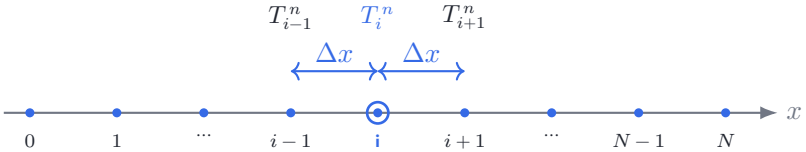
Пространственная сетка

Непрерывную область

$$0 \le x \le L$$

разбиваем на узлы:

$$x_i = i\Delta x, \qquad \Delta x = \frac{L}{N}, \qquad i = 0, 1, \dots, N$$



После дискретизации температурное поле заменяется конечным набором значений:

$$T_i^n \approx T(x_i, t_n)$$

где

i — номер пространственного узла, n — номер шага по времени.

5

Метод конечных разностей для теплопроводности

Аппроксимация производных

Производная по времени

Используем разность вперёд:

$$\left.\frac{\partial T}{\partial t}\right|_i^n \approx \frac{T_i^{n+1} - T_i^n}{\Delta t}$$

Используются два временных слоя:

$$n \quad \to \quad n+1.$$

Вторая производная по пространству

Используем центральную разность:

$$\left.\frac{\partial^2 T}{\partial x^2}\right|_i^n \approx \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2}$$

Используются соседние узлы:

$$i-1, \qquad i, \qquad i+1.$$

После подстановки:

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2}$$

FTCS = Forward Time + Central Space

Явная схема FTCS

Из разностного уравнения выражаем температуру нового временного слоя:

$$T_i^{n+1} = T_i^n + \mathrm{Fo}\left(T_{i+1}^n - 2T_i^n + T_{i-1}^n\right)$$

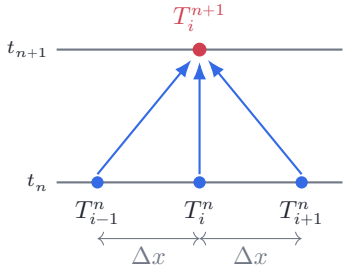
где

$$\mathrm{Fo} = \frac{\alpha \Delta t}{\Delta x^2}$$

— сеточное число Фурье.

Вычисление нового слоя

Все величины в правой части $T_{i-1}^n, \quad T_i^n, \quad T_{i+1}^n$ уже известны.
Поэтому T_i^{n+1} вычисляется напрямую.



$$T_{i-1}^n,\ T_i^n,\ T_{i+1}^n \implies T_i^{n+1}$$

Устойчивость явной схемы

Для одномерного уравнения теплопроводности явная FTCS-схема устойчива при

$$\text{Fo} = \frac{\alpha \Delta t}{\Delta x^2} \leq \frac{1}{2}$$

Следовательно,

$$\Delta t \leq \frac{\Delta x^2}{2\alpha}$$

Сгущаем пространственную сетку

Пусть

$$\Delta x_{\text{new}} = \frac{\Delta x}{10}.$$

Тогда

$$\Delta x^2_{\text{new}} = \frac{\Delta x^2}{100}.$$

Шаг по времени

При том же числе Fo:

$$\Delta t_{\text{new}} = \frac{\Delta t}{100}$$

На том же интервале времени потребуется в 100 раз больше шагов.

Начальные и граничные условия

Начальное условие

Температурное поле в начальный момент времени:

$$T_i^0 = T(x_i, 0)$$

Dirichlet

Задана температура:

$$T(0, t) = T_L.$$

Например:

$$T_L = 100^\circ\text{C}.$$

Neumann

Задан тепловой поток:

$$-k\frac{\partial T}{\partial n} = q_n.$$

Адиабатическая граница:

$$q_n = 0.$$

Robin

Конвективный теплообмен:

$$-k\frac{\partial T}{\partial n} = h(T - T_\infty).$$

Температура границы не задана заранее.

n — внешняя нормаль: слева $-e_x$, справа $+e_x$. $q_n > 0$ означает поток наружу.

Сетка и начальная температура

```
alpha, L, Nx = 1e-4, 0.10, 51
TL, TR = 100.0, 20.0
x = np.linspace(0, L, Nx)
dx = x[1] - x[0]
T_initial = np.full(Nx, 20.0)
T_initial[0], T_initial[-1] = TL, TR

Fo = 0.4
dt = Fo*dx**2/alpha
n_steps = 500
```

Размеры и шаги

$L = 0.10$ м, $N_x = 51$.

$\alpha = 10^{-4}$ м^2/с.

$\Delta x = 0.002$ м.

$\Delta t = 0.016$ с.

500 шагов: $t = 8$ с.

Внутри сначала $20\,^\circ$C. Слева при $t = 0$ включаем нагрев до $100\,^\circ$C.

Явная схема FTCS на Python

```python
def ftcs(T_initial, Fo, n_steps, TL, TR):
    T = np.asarray(T_initial, dtype=float).copy()
    T[0], T[-1] = TL, TR
    history = [T.copy()]
    for n in range(n_steps):
        T_new = T.copy()
        T_new[1:-1] = (T[1:-1] + Fo *
                       (T[2:] - 2*T[1:-1] + T[:-2]))
        T_new[0], T_new[-1] = TL, TR
        T = T_new
        history.append(T.copy())
    return np.array(history)
```

Два временных слоя

`T` хранит старую температуру.

`T_new` хранит новую.

`history` сохраняет копию каждого слоя.

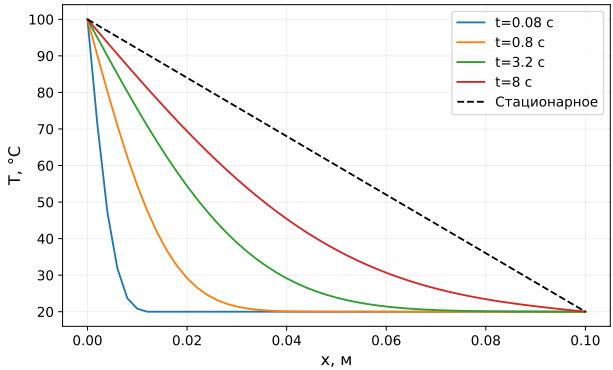
Эволюция температурного профиля

```
history = ftcs(
    T_initial, 0.4,
    500, TL, TR)
T_final = history[-1]
```

При $t = 8$ с:

$T(L/2) \approx 36.9054\,^\circ$C.

Граничные значения сохраняются: 100 и $20\,^\circ$C.

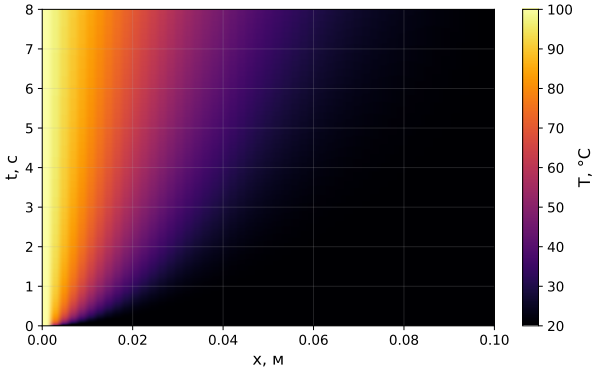


Температурное поле на сетке

Каждая строка `history` — один момент времени.

Каждый столбец — один узел по координате.

Размер массива: 501×51.



FTCS: устойчивый и неустойчивый расчёт

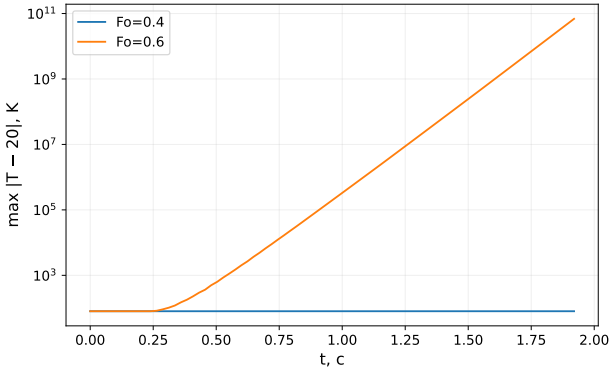
Одна сетка, одинаковые начальные и граничные условия.

Оба расчёта до 1.92 с:

Fo $= 0.4$: 120 шагов.

Fo $= 0.6$: 80 шагов.

При Fo $= 0.6$ возникают растущие колебания.



Проверка FTCS на известном решении

Для отдельного теста обе границы равны $20\,^\circ$C:

$$T(x,t) = 20 + 80 e^{-\alpha (\pi/L)^2 t} \sin \frac{\pi x}{L}.$$

Узлов N_x	Шаг Δx, м	Максимальная ошибка при $t = 8$ с, К
26	0.004	0.052858
51	0.002	0.013217
101	0.001	0.003303

При постоянном Fo $= 0.4$ уменьшение Δx вдвое снижает ошибку примерно в 4 раза.

Явная и неявная схемы

Явная схема

Используются только известные значения слоя n:

$$T_i^{n+1} = \Phi\left(T_{i-1}^n, T_i^n, T_{i+1}^n\right).$$

Поэтому новый слой вычисляется напрямую.

Неявная схема

Пространственные производные берутся на новом слое:

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha \frac{T_{i+1}^{n+1} - 2T_i^{n+1} + T_{i-1}^{n+1}}{\Delta x^2}.$$

Все T_i^{n+1} связаны между собой.

Поэтому на каждом временном шаге возникает система:

$$A\mathbf{T}^{n+1} = \mathbf{b}$$

Матрица неявной схемы

$$-\mathrm{Fo}T_{i-1}^{n+1} + (1 + 2\mathrm{Fo})T_i^{n+1} - \mathrm{Fo}T_{i+1}^{n+1} = T_i^n.$$

$$A = \begin{pmatrix} 1+2\mathrm{Fo} & -\mathrm{Fo} & 0 & \cdots \\ -\mathrm{Fo} & 1+2\mathrm{Fo} & -\mathrm{Fo} & \cdots \\ 0 & \ddots & \ddots & \ddots \\ \vdots & \cdots & -\mathrm{Fo} & 1+2\mathrm{Fo} \end{pmatrix}.$$

При $\mathrm{Fo} = 2$

Главная диагональ: 5.

Соседние диагонали: -2.

Для 51 узла неизвестны 49 внутренних температур. Матрица имеет размер 49×49 и не меняется во времени.

Граничные условия в правой части

$$A\mathbf{T}_{\text{int}}^{n+1} = \mathbf{b}, \qquad \mathbf{b} = \begin{pmatrix} T_1^n + \text{Fo}T_L \\ T_2^n \\ \vdots \\ T_{N_x-2}^n + \text{Fo}T_R \end{pmatrix}.$$

Шаг при известной матрице A

```
b = T[1:-1].copy()
b[0] += Fo * TL
b[-1] += Fo * TR
T[1:-1] = np.linalg.solve(A, b)
```

Граничные температуры известны. Их вклад переносим в правую часть, а решаем систему только для внутренних узлов.

Здесь $T_L = 100\,^\circ\text{C}$, $T_R = 20\,^\circ\text{C}$; при $\text{Fo} = 2$ добавляем 200 и 40.

Неявная схема на Python

```
def backward_euler(T_initial, Fo, n_steps, TL, TR):
    T = np.asarray(T_initial, dtype=float).copy()
    T[0], T[-1] = TL, TR
    n_int = len(T) - 2
    A = (np.diag((1 + 2*Fo)*np.ones(n_int))
         + np.diag(-Fo*np.ones(n_int-1), 1)
         + np.diag(-Fo*np.ones(n_int-1), -1))
    history = [T.copy()]
    for n in range(n_steps):
        b = T[1:-1].copy()
        b[0] += Fo*TL
        b[-1] += Fo*TR
        T[1:-1] = np.linalg.solve(A, b)
        T[0], T[-1] = TL, TR
        history.append(T.copy())
    return np.array(history)
```

Обратный Эйлер

Матрицу A строим один раз.

На каждом шаге обновляем **b** и решаем систему.

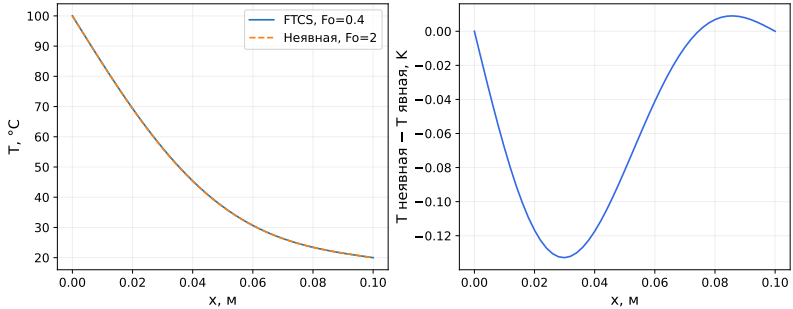
Для этой линейной задачи схема устойчива при любом $\Delta t > 0$.

Сравнение схем при $t = 8$ с

```
implicit_history = backward_euler(
    T_initial, 2.0, 100, TL, TR)
```

FTCS: $\Delta t = 0.016$ с, 500 шагов.

Неявная: $\Delta t = 0.08$ с, 100 шагов.



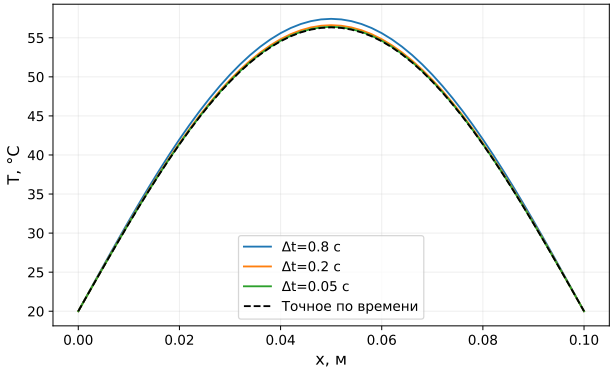
Максимальное расхождение 0.133 К. Это разница двух приближений, а не известная точная ошибка.

Большой шаг в неявной схеме

Синусоидальный тест, обе границы $20\,^{\circ}$C.

При $t = 8$ с сравниваем с точным по времени решением на той же пространственной сетке.

Все расчёты устойчивы, но большой шаг сильнее искажает скорость охлаждения.

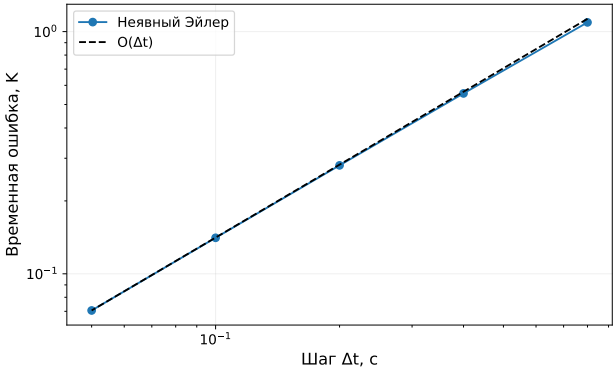


Временная ошибка неявного Эйлера

Сравниваем с точным по времени решением системы после пространственной дискретизации.

Так отделяем временную ошибку от пространственной.

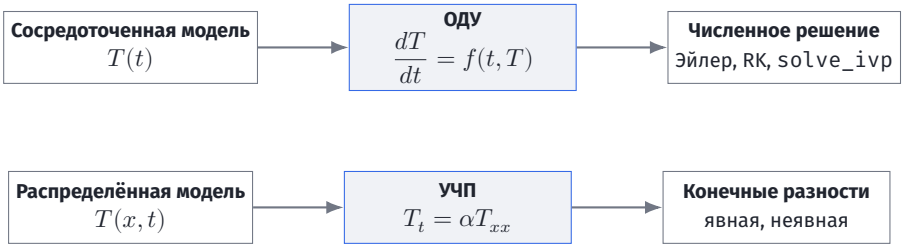
Уменьшение шага вдвое снижает ошибку примерно в 2 раза.



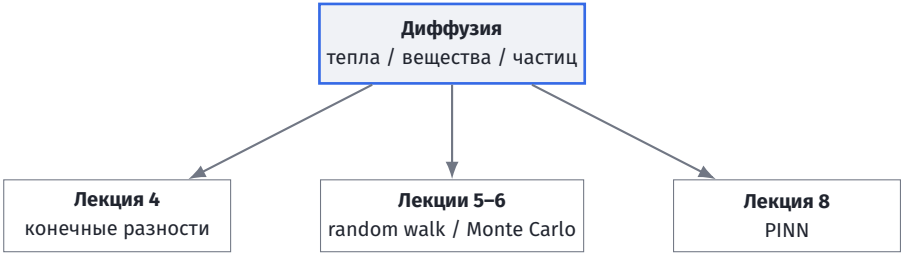
6

Практика и итоги

Модели теплообмена



Методы моделирования диффузии



Итоги лекции

- **ОДУ:** методы Эйлера, RK4 и адаптивный `solve_ivp`. Проверяем решение по точной кривой или балансу энергии.
- **Теплопроводность:** FTCS обновляет слой напрямую, неявная схема решает линейную систему.
- **Шаг сетки:** проверяем устойчивость и изменение ошибки при уменьшении шага.

Код занятия

github.com/lyushisyan/computational-methods-course